

Design and Implementation of a Mini-RISC CPU Simulator

Name: Siddharth Narayan Mishra **Roll Number:** 123CS0189

1. Introduction

Modern processors execute programs by repeatedly fetching instructions from memory, decoding them, performing the required computation, and storing the results. Although this process happens billions of times per second in real hardware, its internal working is hidden from the programmer.

The objective of this project is to build a **software simulator** of a small RISC-style processor so that the internal working of a CPU can be clearly observed. The simulator models registers, memory, flags, and the instruction execution pipeline, and produces a detailed execution trace for every instruction executed.

The CPU implemented in this project follows a minimal architecture specified in the lab assignment. It includes four general-purpose registers, an accumulator, flags, and a simple instruction set that supports arithmetic, comparisons, and control flow.

2. CPU Architecture

2.1 Registers

The simulated CPU contains the following registers:

Register	Description
R0–R3	Four 8-bit general purpose registers
ACC	8-bit accumulator
PC	Program counter
IR	Instruction register

Register	Description
Z	Zero flag
C	Carry flag

All arithmetic and logical operations use the accumulator as both input and output.

2.2 Memory

The CPU has **256 bytes of memory**, which stores the program instructions. Each memory location contains one 8-bit instruction.

2.3 Instruction Format

Each instruction is 8 bits long and is divided into two parts:

```
[ OPCODE (4 bits) ][ OPERAND (4 bits) ]
```

The opcode determines the operation, and the operand specifies either a register number or a jump address.

2.4 Instruction Set

Opcode	Mnemonic	Operation
0×0	NOP	No operation
0×1	LOAD Rn	$ACC \leftarrow Rn$
0×2	STORE Rn	$Rn \leftarrow ACC$
0×3	MOVI k	$ACC \leftarrow k$
0×4	ADD Rn	$ACC \leftarrow ACC + Rn$
0×5	SUB Rn	$ACC \leftarrow ACC - Rn$
0×6	MUL Rn	$ACC \leftarrow ACC \times Rn$

Opcode	Mnemonic	Operation
0x7	DIV Rn	$ACC \leftarrow ACC \div Rn$
0x8	POW Rn	$ACC \leftarrow ACC \wedge Rn$
0x9	CMP Rn	Set flags from $ACC - Rn$
0xA	JMP addr	$PC \leftarrow addr$
0xB	JZ addr	Jump if $Z = 1$
0xC	JC addr	Jump if $C = 1$
0xF	HALT	Stop execution

3. Execution Cycle

Each instruction is executed using a four-step cycle:

- Fetch**

The instruction at the memory address pointed to by PC is loaded into IR.

- Decode**

The opcode and operand are extracted from the instruction.

- Execute**

The ALU or control logic performs the operation.

- Update**

The accumulator, registers, flags, and PC are updated.

This process continues until a HALT instruction is encountered.

4. Software Design

The simulator is implemented in **C++**. The CPU is represented using a class that contains all registers and flags. Memory is implemented as a 256-byte array. The design is modular and includes:

- A fetch unit

- An execution unit
- An instruction decoder
- A program loader
- A logging system

This structure closely follows how real CPUs are organized internally.

5. Execution Trace

After each instruction is executed, the simulator logs the CPU state to `cpu_trace.txt` in the following format:

```
[TIME] PC=05 IR=0x61 INST=MUL R1 | R0=5 R1=3 R2=0 R3=0 | ACC=15 | Z=0 C=0
```

This trace shows:

- The current time
- The program counter
- The raw instruction in hex
- The decoded instruction
- Register values
- Accumulator
- Zero and Carry flags

This allows step-by-step verification of program execution.

6. Sample Programs

Program 1: Multiply 7×4

```
MOVI 7
```

```
STORE R0
MOVI 4
MUL R0
HALT
```

Result: ACC = 28

Program 2: $(5 + 6) \times 3$

```
MOVI 5
STORE R0
MOVI 6
ADD R0
STORE R0
MOVI 3
MUL R0
HALT
```

Result: ACC = 33

Program 3: Compare 5 and 5

```
MOVI 5
STORE R0
MOVI 5
CMP R0
HALT
```

Result: Z = 1

7. How to Compile and Test

Folder Structure

```
cpu.cpp  
cpu  
cpu_trace.txt  
mul.hex  
expr.hex  
cmp.hex
```

Compile

```
g++ cpu.cpp -o cpu
```

Run

```
./cpu
```

When prompted, enter a program file name such as:

```
mul.hex
```

View Output

After execution, open:

```
cpu_trace.txt
```

to view the full instruction-by-instruction trace.

8. Conclusion

This project successfully demonstrates the internal working of a simple RISC-style processor. The simulator models registers, memory, and flags, and accurately executes programs using the fetch–decode–execute cycle. The execution trace provides detailed insight into how each instruction affects the CPU state, making the simulator a valuable learning tool for understanding computer architecture.